17.02.2026

Раздельная компиляция. Объектные файлы. BLAS, LAPACK, MKL. Работа с простыми числами

Филиппов Михаил Витальевич

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2025-2026

Новосибирский государственный университет

*НАСТОЯЩАЯ НАУКА

Давайте познакомимся

Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Являюсь м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++
- к.ф.-м.н.

Новосибирский государственный университет
*НАСТОЯЩАЯ НАУКА

План лекции

Раздельная компиляция	Объектные файлы	BLAS, LAPACK, MKL	Работа с простыми числами
30 минут	25 минут	20 минут	15 минут

План лекции

Раздельная компиляция	Объектные файлы	BLAS, LAPACK, MKL	Работа с простыми числами
30 минут	25 минут	20 минут	15 минут

Общие слова

Компиляция программ, состоящих из двух и более файлов исходного кода

Unix
Компиляция a.out: cc file1.c file2.c -> file1.o и file2.o

Linux
Компиляция a.out: gcc file1.c file2.c -> file1.o и file2.o

Компиляторы командной строки DOS
Аналогично, но .o -> .obj

Хорошим тоном в программировании считается размещение прототипов функций и определенных констант в заголовочном файле.

Компиляторы интегрированных сред разработки (integrated development environment — IDE) для операционных систем Windows и Macintosh являются сориентированными на проекты. Проект описывает ресурсы, используемые конкретной программой.

Использование заголовочных файлов

- Если поместить функцию main() в один файл, а определения собственных функций — в другой, то в первом файле по-прежнему будут нужны прототипы функций. Вместо того чтобы набирать их каждый раз, когда применяется файл с функциями, прототипы функций можно сохранить в заголовочном файле.
- Более разумное решение предполагает размещение директив #define в заголовочном файле с последующим использованием директивы #include в каждом файле исходного кода.
- При разделении на h и c файлы повышается читаемость кода и эффективность компиляции.

Демо – несколько файлов: hotel.h

```
/* hotel.h -- константы и объявления для программы hotel.c */
#define QUIT 5
#define HOTEL1 180.00
#define HOTEL2 225.00
#define HOTEL3 255.00
#define HOTEL4 355.00
#define DISCOUNT 0.95
#define STARS "*************************************"
// отображает список возможных вариантов
int menu(void);
// возвращает количество суток, на которое бронируется номер
int getnights(void);
// вычисляет стоимость в зависимости от расценок и количества
// забронированных суток и отображает результат
void showprice(double rate, int nights);
```

Демо – несколько файлов: hotel.c 1/2

```
/* hotel.c – функции управления отелем */
#include <stdio.h>
#include "hotel.h"
int menu(void)
{
    int code, status;
    printf("\n%s%s\n", STARS, STARS);
    printf("Введите число, соответствующее выбранному отелю:\n");
    printf("1) Fairfield Arms 2) Hotel Olympic\n");
    printf("3) Chertworthy Plaza 4) The Stockton\n");
    printf("5) выход \n");
    printf("%s%s\n", STARS, STARS);
    while ((status = scanf("%d", &code)) != 1 || (code < 1 || code > 5))
    {
        if (status != 1)
            scanf("%*s"); // отбрасывание нецелочисленного ввода
        printf("Введите целое число от 1 до 5.\n");
    }
    return code;
}
```

Демо – несколько файлов: hotel.c 2/2

```
int getnights(void)
{
    int nights;
    printf("На сколько суток вы бронируете номер? ");
    while (scanf("%d", &nights) != 1)
    {
        scanf("%*s"); // исключение нецелочисленного ввода
        printf("Введите целое число, такое как 2.\n");
    }
    return nights;
}
void showprice(double rate, int nights)
{
    int n;
    double total = 0.0;
    double factor = 1.0;
    for (n = 1; n <= nights; n++, factor *= DISCOUNT)
        total += rate * factor;
    printf("Общая стоимость составляет $%0.2f.\n", total);
}
```

Демо – несколько файлов: main.c

```c
#include <stdio.h>
#include "hotel.h" /* определяет константы, объявляет функции */
int main(void)
{
    int nights, code; double hotel_rate;
    while ((code = menu()) != QUIT)
    {
        switch (code)
        {
        case 1:
            hotel_rate = HOTEL1;
            break;
        …
        case 4:
            hotel_rate == HOTEL4;
            break;
        default:
            hotel_rate = 0.0;
            printf("Ошибка!\n");
            break;
        }
        nights = getnights();
        showprice(hotel_rate, nights);
    }
    return 0;
```

```
*****************************************************************
Введите число, соответствующее выбранному отелю:
I) Fairfield Arms 2) Hotel Olympic
3) Chertworthy Plaza 4) The Stockton
5) выход
*****************************************************************
1
На сколько суток вы бронируете номер? 5
Общая стоимость составляет $814.39.

*****************************************************************
Введите число, соответствующее выбранному отелю:
I) Fairfield Arms 2) Hotel Olympic
3) Chertworthy Plaza 4) The Stockton
5) выход
*****************************************************************
4
На сколько суток вы бронируете номер? 5
Общая стоимость составляет $1153.72.

*****************************************************************
Введите число, соответствующее выбранному отелю:
I) Fairfield Arms 2) Hotel Olympic
3) Chertworthy Plaza 4) The Stockton
5) выход
*****************************************************************
5
Благодарим за использование и желаем успехов.
```

Копнем зачем и как

Процесс компиляции

Компиляция файлов, написанных на языке C, обычно длится лишь несколько секунд, но за это время исходный код успевает пройти процесс обработки с участием четырех отдельных компонентов, каждый из которых имеет определенное назначение:

- препроцессор;
- компилятор;
- ассемблер;
- компоновщик.

Каждый из них принимает определенный ввод от предыдущего компонента и генерирует определенный вывод для следующего. Этот процесс продолжается, пока последний компонент не сгенерирует итоговый продукт.

Полный список доступных компиляторов можно найти в «Википедии»: https://en.wikipedia.org/wiki/List_of_compilers#C_compilers

Процесс компиляции

Платформа — сочетание операционной системы и оборудования (или архитектуры), самая важная часть которого — набор инструкций центрального процессора. Операционная система играет роль программного компонента платформы, а аппаратный компонент определяется архитектурой. Например, мы можем иметь дело с ОС Ubuntu на ARM-плате или с Microsoft Windows на компьютере с 64-битным процессором AMD.

Кросс-платформенное программное обеспечение может работать на разных платформах. Но при этом необходимо понимать, что кросс-платформенность отличается от переносимости. Кросс-платформенное ПО обычно предоставляет разные двоичные (итоговые объектные) файлы и установщики для каждой среды, тогда как переносимые программы везде используют одни и те же исполняемые и установочные файлы.

Некоторые компиляторы для C, такие как gcc и clang, являются кросс-платформенными — они умеют генерировать код для разных платформ.

Сборка проекта на языке С

Заголовочные и исходные файлы

Любой проект на С содержит исходный код (или кодовую базу) и другие документы, которые описывают разрабатываемое приложение и используемые стандарты. Код С обычно хранится в файлах двух типов:

- в заголовочных файлах, как правило, имеющих расширение .h;
- в исходных файлах с расширением.с.

Для краткости заголовочные файлы называются заголовками, а исходные файлы — исходниками.

Сборка проекта на языке C

- Заголовочный файл обычно содержит перечисления, макросы и определения типов, а также объявления функций, глобальных переменных и структур. В языке C объявление и определение некоторых элементов программирования, таких как функции, переменные и структуры, могут находиться в разных файлах.
- Объявления принято хранить в заголовочных файлах, а соответствующие определения — в исходных. Это особенно относится к функциям.
- Объявления функций настоятельно рекомендуется размещать в заголовках, а их определения — в соответствующих исходниках. И хотя это не является обязательным требованием, такой подход к проектированию позволит вам хранить определения функций вне заголовочных файлов.
- Определения структур и объявления можно хранить и в разных файлах, но это делается в особых случаях.
- К заголовочным файлам можно подключать только другие заголовки, но не исходники. К исходным файлам можно подключать только заголовки. Подключение одних заголовочных файлов к другим считается дурным тоном. Если вы так делаете, то это обычно говорит о серьезной проблеме в архитектуре вашего проекта.

Сборка проекта на языке С

объявление функции

```
double average(int *, int);
```

определение функции

```
double average(int *array, int length)
{
    if (length <= 0)
        return 0;
    double sum = 0.0;
    for (int i = 0; i < length; i++)
        sum += array[i];
    return sum / length;
}
```

Правило 1: компилируются только исходные файлы. Заголовки не должны содержать ничего, кроме объявлений.
Правило 2: каждый исходный файл компилируется по отдельности.

!!!!!!Объявление функции хранится в заголовочном файле, а определение (или тело) — в исходном. Нарушать данное правило можно лишь в редких случаях. Кроме того, чтобы иметь доступ к объявлению, исходник должен подключить заголовочный файл.

Пример 2

main.c

```c
#include <stdio.h>
#include "average.h"
int main(int argc, char **argv)
{
    // объявление массива
    int array[5];
    // заполнение массива
    array[0] = 10;
    array[1] = 3;
    array[2] = 5;
    array[3] = -8;
    array[4] = 9;
    // вычисление среднего значения с
помощью функции avg
    double average = avg(array, 5,
NORMAL);
    printf("The average: %f\n",
average);
    average = avg(array, 5, SQUARED);
    printf("The squared average: %f\n",
average);
    return 0;
}
```

average.h

```c
#ifndef AVERAGE_H
#define AVERAGE_H
typedef enum
{
    NONE,
    NORMAL,
    SQUARED
} average_type_t;
// объявление функции
double avg(int *, int, average_type_t);
#endif
```

average.c

```c
#include "average.h"
double avg(int *array, int length,
average_type_t type)
{
    if (length <= 0 || type == NONE)
        return 0;
    double sum = 0.0;
    for (int i = 0; i < length; i++)
    {
        if (type == NORMAL)
            sum += array[i];
        else if (type == SQUARED)
            sum += array[i] * array[i];
    }
    return sum / length;
}
```

Этап 1: предобработка

Это первый этап компиляции. К исходному файлу подключается ряд заголовков. Но перед компиляцией препроцессор собирает их содержимое в единый блок кода. Иными словами, после предобработки мы получаем один фрагмент кода на языке C, сформированный путем копирования заголовочных файлов в исходные.

На данном этапе должны быть выполнены и другие директивы препроцессора. Код, прошедший такую обработку, называется единицей трансляции (или единицей компиляции). **Единица трансляции** — это отдельный логический блок кода на C, сгенерированный препроцессором и готовый к компиляции.

```
$ gcc -E average.c > average.i
```

average.i

average.i > ...

```
1   # 0 "average.c"
2   # 0 "<built-in>"
3   # 0 "<command-line>"
4   # 1 "/usr/include/stdc-predef.h" 1 3 4
```

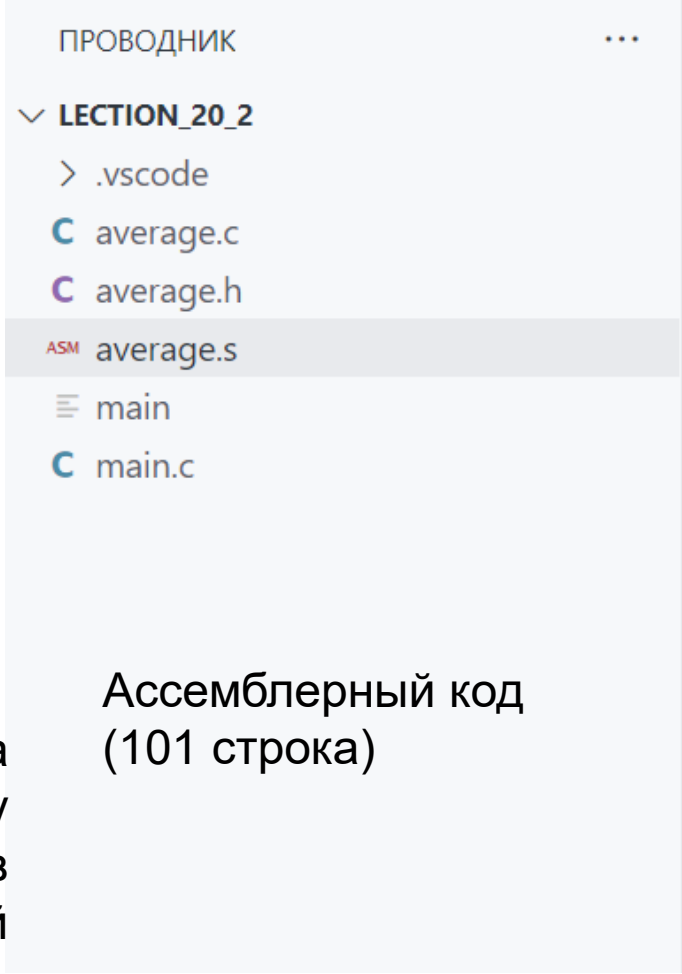
```
lection_20_2> gcc -E average.c
# 0 "average.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "average.c"
# 1 "average.h" 1

typedef enum
{
    NONE,
    NORMAL,
    SQUARED
} average_type_t;

double avg(int *, int, average_type_t);
# 2 "average.c" 2
double avg(int *array, int length,
average_type_t type)
{
    if (length <= 0 || type == NONE)
        return 0;
    double sum = 0.0;
    for (int i = 0; i < length; i++)
    {
        if (type == NORMAL)
            sum += array[i];
        else if (type == SQUARED)
            sum += array[i] * array[i];
    }
    return sum / length;
}
```

Этап 2: компиляция в ассемблерный код

```
$ gcc -S average.c
$ cat average.s
```

После получения единицы трансляции можно переходить ко второму этапу — компиляции. На вход подается единица компиляции, полученная на предыдущем этапе, а на выходе получается соответствующий ассемблерный код. Он все еще может быть прочитан человеком, но уже зависит от аппаратной архитектуры и приближен к оборудованию. Для превращения в машинные инструкции его нужно продолжать обрабатывать.

В рамках данного этапа компилятор анализирует единицу трансляции и превращает ее в ассемблерный код, рассчитанный на ***целевую архитектуру***.



Ассемблерный код (101 строка)

average.s

average.s

```
1        .file   "average.c"
2        .text
3        .globl  avg
4        .type   avg, @function
5    avg:
6    .LFB0:
7        .cfi_startproc
8        endbr64
9        pushq   %rbp
10       .cfi_def_cfa_offset 16
11       .cfi_offset 6, -16
12       movq    %rsp, %rbp
13       .cfi_def_cfa_register 6
14       movq    %rdi, -24(%rbp)
15       movl    %esi, -28(%rbp)
16       movl    %edx, -32(%rbp)
```

Этап 3: компиляция в машинные инструкции

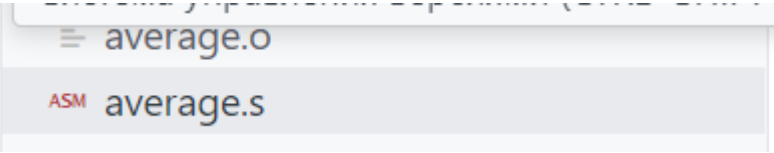
Цель — сгенерировать инструкции машинного уровня (или машинный код) на основе ассемблерного кода, созданного компилятором на предыдущем этапе. У каждой архитектуры есть свой ассемблер, который может преобразовать собственный ассемблерный код в машинные инструкции.

Файл с машинными инструкциями, который мы сгенерируем в этом подразделе, называется ***объектным файлом***. Переносимые объектные файлы иногда называют промежуточными.

Цель этапа — сгенерировать переносимый объектный файл из ассемблерного кода, созданного компилятором. Любой другой созданный нами продукт будет основан на объектных файлах, сгенерированных ассемблером на данном этапе.

Создание объектного файла из ассемблерного кода

```
$ as average.s -o average.o
```



Компиляция одного из исходников с последующей генерацией переносимого объектного файла

```
$ gcc -c main.c
```

Двоичный файл и объектный файл содержат машинные инструкции и являются синонимами. Как можно видеть в приведенной выше команде, имя выходного объектного файла указывается с помощью параметра -o. Переносимые объектные файлы обычно имеют расширение .o (или .obj в Microsoft Windows). Содержимое объектного файла, будь то .o или .obj, нельзя представить в текстовом виде.

Этап 4: компоновка

На данном этапе необходимо объединить два объектных файла, чтобы создать еще один объектный файл, на сей раз исполняемый. Это делается путем компоновки.

Поддержка новых архитектур

Можно собрать для новой архитектуры, если выполнены два условия.

1. Известна версия ассемблера.
2. Доступна утилита (или программа) от соответствующего производителя, позволяющая скомпилировать ассемблерный код в машинные инструкции.

Двумя инструментами, без которых нельзя обойтись при работе с новой архитектурой, являются:

- компилятор языка C;
- компоновщик.

Попытка скомпоновать объектные файлы путем ручного использования утилиты ld

```
$ ld average.o main.o
ld: warning: cannot find entry symbol
_start; defaulting to 0000000000401000
ld: main.o: in function `main':
main.c:(.text+0x7d): undefined
reference to `printf'
ld: main.c:(.text+0xb9): undefined
reference to `printf'
ld: main.c:(.text+0xd2): undefined
reference to `__stack_chk_fail'
```

Использование gcc для компоновки объектных файлов

```
$ gcc average.o main.o
$ ./a.out
The average: 3.800000
The squared average: 55.800000
```

Подробное описание этапа

Для работы платформам, аналогичным Unix-подобным операционным системам, нужны ранее упомянутые инструменты, такие как ассемблер и компоновщик. Компоновщик по умолчанию в Unix-подобных системах — ld.

Ошибка, нам нужно предоставить компоновщику какие-то другие объектные файлы (не обязательно переносимые), содержащие определения функций printf и __stack_chk_fail.

Более простой способ создания итогового исполняемого файла. Сборка проекта состоит из компиляции исходников с последующей их компоновкой и, возможно, добавлением других библиотек. В результате получаются итоговые продукты.

Препроцессор

Как работает препроцессор: копирует содержимое другого файла или развертывает макрос, заменяя текст.

Препроцессор ничего не знает о языке C; прежде чем выполнять какие-либо дальнейшие действия, он должен разобрать входящий файл с помощью синтаксического анализатора. То есть использует анализатор для поиска директив в исходном коде.

Синтаксический анализатор — это, как правило, программа, которая разбирает входные данные и извлекает из них определенные элементы для дальнейшего анализа и обработки. Чтобы разбить входные данные на более мелкие и полезные элементы, анализатор должен понимать их структуру.

В большинстве Unix-подобных операционных систем есть утилита под названием cpp (расшифровывается как C Pre-Processor, а не как C Plus Plus!). Она входит в пакет разработки для языка C, который поставляется вместе с любой разновидностью Unix. Утилита позволяет предварительно обрабатывать файлы на C, что, собственно, и делают в фоновом режиме такие компиляторы, как gcc.

```
$ cpp average.c
# 0 "average.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "average.c"
# 1 "average.h" 1

typedef enum
{
    NONE,
    NORMAL,
    SQUARED
} average_type_t;

double avg(int *, int, average_type_t);
# 2 "average.c" 2
double avg(int *array, int length,
average_type_t type)
{
    if (length <= 0 || type == NONE)
        return 0;
    double sum = 0.0;
    for (int i = 0; i < length; i++)
    {
        if (type == NORMAL)
            sum += array[i];
        else if (type == SQUARED)
            sum += array[i] * array[i];
    }
    return sum / length;
}
```

Компилятор

Компилятор принимает на вход единицу трансляции, подготовленную препроцессором, и генерирует соответствующие инструкции ассемблера. После компиляции множественных исходников на языке C начинается преобразование сгенерированного ассемблерного кода в переносимые объектные файлы, которые затем объединяются (возможно, с помощью других объектных файлов) в библиотеку или исполняемую программу; при этом применяются такие инструменты, как ассемблер и компоновщик, входящие в состав платформы.

Одна из сложностей компиляции кода на C состоит в получении корректных ассемблерных инструкций, совместимых с целевой архитектурой. Утилита gcc позволяет компилировать один и тот же исходник для разных архитектур (пример. ARM, Intel x86, AMD и др). У каждой архитектуры есть свой набор инструкций процессора, ***полная ответственность за генерацию корректного ассемблерного кода для конкретной архитектуры лежит на компиляторе gcc*** (или другом).

Чтобы преодолеть указанные трудности, gcc (или любой другой компилятор) разделяет данную задачу на два этапа. Вначале он анализирует единицу трансляции и переводит ее в переносимую структуру данных под названием **«дерево абстрактного синтаксиса» (abstract syntax tree, AST);** данная структура не имеет прямого отношения к языку C. Затем на ее основе генерируется подходящий **ассемблерный код** для целевой архитектуры. Все, что относится к определенной архитектуре, происходит на втором этапе. За первый этап отвечает подкомпонент интерфейс компилятора (compiler frontend), а за второй — кодогенератор (compiler backend).

AST

```c
int main() {
    int var1 = 1;
    double var2 = 2.5;
    int var3 = var1 + var2;
    return 0;
}
```

В качестве абстрактной структуры данных, которая передается между интерфейсом компилятора и кодогенератором, проект LLVM Compiler Infrastructure Project использует промежуточное представление (intermediate representation, LLVM IR).

-FunctionDecl представляет функцию main. Выше находится метаинформация о единице трансляции, которая передается компилятору.

Вслед за FunctionDecl идут элементы (или узлы) дерева для объявлений, бинарных операций, инструкции return и даже для выражений неявного приведения типов.

Преимущество генерации AST для исходного кода состоит в том, что вы можете переставить местами инструкции, удалить неиспользуемые ветки и заменить некоторые блоки в целях повышения производительности, не меняя при этом логику программы.

```
erPoint/lection_20_3$ clang -Xclang -ast-dump -fsyntax-only main.c
TranslationUnitDecl 0x9638e8 <<invalid sloc>> <invalid sloc>
|-TypedefDecl 0x964110 <<invalid sloc>> <invalid sloc> implicit __int128_t '__int128
| `-BuiltinType 0x963eb0 '__int128'
|-TypedefDecl 0x964180 <<invalid sloc>> <invalid sloc> implicit __uint128_t 'unsigne
| `-BuiltinType 0x963ed0 'unsigned __int128'
|-TypedefDecl 0x964488 <<invalid sloc>> <invalid sloc> implicit __NSConstantString '
stantString_tag'
| `-RecordType 0x964260 'struct __NSConstantString_tag'
|   `-Record 0x9641d8 '__NSConstantString_tag'
|-TypedefDecl 0x964520 <<invalid sloc>> <invalid sloc> implicit __builtin_ms_va_list
| `-PointerType 0x9644e0 'char *'
|   `-BuiltinType 0x963990 'char'
|-TypedefDecl 0x964818 <<invalid sloc>> <invalid sloc> implicit __builtin_va_list 's
t_tag[1]'
| `-ConstantArrayType 0x9647c0 'struct __va_list_tag[1]' 1
|   `-RecordType 0x964600 'struct __va_list_tag'
`-CompoundStmt 0x9ba8b0 <line:2:1, line:7:1>
  |-DeclStmt 0x9ba658 <line:3:5, col:17>
  | `-VarDecl 0x9ba5d0 <col:5, col:16> col:9 used var1 'int' cinit
  |   `-IntegerLiteral 0x9ba638 <col:16> 'int' 1
  |-DeclStmt 0x9ba710 <line:4:5, col:22>
  | `-VarDecl 0x9ba688 <col:5, col:19> col:12 used var2 'double' cinit
  |   `-FloatingLiteral 0x9ba6f0 <col:19> 'double' 2.500000e+00
  |-DeclStmt 0x9ba868 <line:5:5, col:27>
  | `-VarDecl 0x9ba740 <col:5, col:23> col:9 var3 'int' cinit
  |   `-ImplicitCastExpr 0x9ba850 <col:16, col:23> 'int' <FloatingToIntegral>
  |     `-BinaryOperator 0x9ba830 <col:16, col:23> 'double' '+'
  |       |-ImplicitCastExpr 0x9ba818 <col:16> 'double' <IntegralToFloating>
  |       | `-ImplicitCastExpr 0x9ba7e8 <col:16> 'int' <LValueToRValue>
  |-DeclStmt 0x9ba658 <line:3:5, col:17>
  | `-VarDecl 0x9ba5d0 <col:5, col:16> col:9 used var1 'int' cinit
  |   `-IntegerLiteral 0x9ba638 <col:16> 'int' 1
  |-DeclStmt 0x9ba710 <line:4:5, col:22>
  | `-VarDecl 0x9ba688 <col:5, col:19> col:12 used var2 'double' cinit
  |   `-FloatingLiteral 0x9ba6f0 <col:19> 'double' 2.500000e+00
  |-DeclStmt 0x9ba868 <line:5:5, col:27>
  | `-VarDecl 0x9ba740 <col:5, col:23> col:9 var3 'int' cinit
  |   `-ImplicitCastExpr 0x9ba850 <col:16, col:23> 'int' <FloatingToIntegral>
  |     `-BinaryOperator 0x9ba830 <col:16, col:23> 'double' '+'
  |       |-ImplicitCastExpr 0x9ba818 <col:16> 'double' <IntegralToFloating>
  |       | `-ImplicitCastExpr 0x9ba7e8 <col:16> 'int' <LValueToRValue>
  |       |   `-DeclRefExpr 0x9ba7a8 <col:16> 'int' lvalue Var 0x9ba5d0 'var1' 'int'
  |       `-ImplicitCastExpr 0x9ba800 <col:23> 'double' <LValueToRValue>
  |         `-DeclRefExpr 0x9ba7c8 <col:23> 'double' lvalue Var 0x9ba688 'var2' 'double'
  `-ReturnStmt 0x9ba8a0 <line:6:5, col:12>
    `-IntegerLiteral 0x9ba880 <col:12> 'int' 0
```

Ассемблер

Для создания объектных файлов с подходящими машинными инструкциями платформа должна предоставлять ассемблер. В Unix-подобных операционных системах ассемблер можно использовать с помощью специальной утилиты.

Если установить на том же компьютере другую Unix-подобную операционную систему, то версия ассемблера может измениться. Это очень важно, поскольку вы можете получить другие объектные файлы, хотя аппаратное обеспечение и соответствующие машинные инструкции останутся прежними!

Объектные файлы, сгенерированные в Linux для 64-битной архитектуры AMD, могут отличаться от результатов компиляции той же программы в другой ОС, такой как FreeBSD или macOS, и на том же оборудовании. Это значит, что несмотря на одинаковые машинные инструкции, объектные файлы не могут совпадать ввиду различных форматов, используемых в разных системах.

Иными словами, каждая ОС поддерживает свой двоичный формат (формат объектных файлов) для хранения инструкций машинного уровня. Поэтому содержимое объектного файла определяется двумя факторами: архитектурой (аппаратным обеспечением) и операционной системой. Эту комбинацию мы обычно называем платформой.

В Linux используется формат ELF (Executable and Linking Format — формат исполняемых и компонуемых файлов). Он применяется для всех исполняемых/объектных файлов и разделяемых библиотек. Проще говоря, в Linux ассемблер создает ELF-файлы.

Компоновщик

Сборка проекта на языке C начинается с компиляции всех его исходников в соответствующие переносимые объектные файлы. Данный этап необходим для создания конечных продуктов, но одного его недостаточно. Остается сделать еще один шаг. Из проекта на C/C++ можно получить следующие продукты:

- ряд исполняемых файлов, которые в большинстве Unix-подобных систем имеют расширение .out. В Microsoft Windows, как правило, используется расширение .exe;
- ряд статических библиотек с расширением .a (в большинстве Unix-подобных систем) или .lib (в Microsoft Windows);
- ряд динамических библиотек или разделяемых объектных файлов. В Unix-подобных операционных системах они обычно имеют расширение .so, а в macOS и Microsoft Windows — .dylib и .dll соответственно.

Переносимые объектные файлы не входят в число этих продуктов. Они представляют собой промежуточные ресурсы, которые используются на этапе компоновки для получения перечисленных выше продуктов; далее они не нужны. Вся ответственность за создание конечных продуктов из переносимых объектных файлов ложится на компоновщик.

Последнее, но важное замечание относится к терминам, которые мы здесь используем: все три продукта называются объектными файлами. Поэтому объектные файлы, сгенерированные ассемблером в качестве промежуточных ресурсов, лучше называть переносимыми.

Принцип работы компоновщика

Компоновщик объединяет все переносимые объектные файлы вдобавок к указанным статическим библиотекам, чтобы получить готовую программу.

Объектный файл содержит инструкции машинного уровня, эквивалентные единице трансляции. Однако они хранятся не в произвольном порядке, а сгруппированы в так называемые символы.

На самом же деле в объектном файле хранится много элементов, но символы — компонент, который объясняет принцип работы компоновщика и то, как из нескольких объектных файлов получается один.

example.c

```c
int average(int a, int b)
{
    return (a + b) / 2;
}
int sum(int *numbers, int count)
{
    int sum = 0;
    for (int i = 0; i < count; i++)
    {
        sum += numbers[i];
    }
    return sum;
}
```

Принцип работы компоновщика

Компиляция исходного файла

```
$ gcc -c example.c -o example.o
```

Использование утилиты nm для просмотра символов, определенных в переносимом объектном файле

```
$ nm example.o
0000000000000000 T average
0000000000000021 T sum
```

Применение утилиты readelf для просмотра таблицы символов в переносимом объектном файле

```
$ readelf -s example.o

Symbol table '.symtab' contains 5 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS example.c
     2: 0000000000000000     0 SECTION LOCAL  DEFAULT    1 .text
     3: 0000000000000000    33 FUNC    GLOBAL DEFAULT    1 average
     4: 0000000000000021    73 FUNC    GLOBAL DEFAULT    1 sum
```

Принцип работы компоновщика

Применение утилиты objdump для просмотра инструкций символов, определенных в переносимом объектном файле

```
$ objdump -d example.o

example.o:     file format elf64-x86-64


Disassembly of section .text:

0000000000000000 <average>:
   0:   f3 0f 1e fa             endbr64
   4:   55                      push   %rbp
   5:   48 89 e5                mov    %rsp,%rbp
   8:   89 7d fc                mov    %edi,-0x4(%rbp)
   b:   89 75 f8                mov    %esi,-0x8(%rbp)
   e:   8b 55 fc                mov    -0x4(%rbp),%edx
  ...
```

Принцип работы компоновщика

decls.h

```
#ifndef DECLS_H
#define DECLS_H
int add(int, int);
int multiply(int, int);
#endif
```

add.c

```
#include "decls.h"
int add(int a, int b) {
    return a + b;
}
```

multiply.c

```
#include "decls.h"
int multiply(int a, int b) {
    return a * b;
}
```

main.c

```
#include "decls.h"
int main(int argc, char **argv)
{
    int x = add(4, 5);
    int y = multiply(9, x);
    return 0;
}
```

Пример состоит из четырех файлов: трех исходников и одного заголовка. В заголовочном файле объявлено две функции, определения которых находятся в разных исходных файлах. Третий исходник содержит функцию main.

Функции, представленные в примере, крайне просты, и после компиляции каждая из них будет состоять всего из нескольких машинных инструкций.

Принцип работы компоновщика

```
$ gcc -c add.c -o add.o
$ gcc -c multiply.c -o multiply.o
$ gcc -c main.c -o main.o
```

Вывод символов, определенных в файле add.o

```
$ nm add.o
0000000000000000 T add
```

Вывод символов, определенных в файле multiply.o

```
$ nm multiply.o
0000000000000000 T multiply
```

Вывод символов, определенных в файле main.o

```
$ nm main.o
                 U add
0000000000000000 T main
                 U multiply
```

U (unresolved — «неразрешенная»)

Принцип работы компоновщика

Подытожим: мы получили три объектных файла, в одном из которых есть неразрешенные символы. Таким образом, перед нами стоит вполне понятная задача: предоставить компоновщику символы, которые можно найти в других объектных файлах. Получив все необходимые символы, компоновщик сможет продолжить их объединение в итоговый рабочий исполняемый файл. Если компоновщику не удастся найти определения неразрешенных символов, то он прервет работу и проинформирует нас об ошибке компоновки.

```
$ gcc add.o multiply.o main.o
```

Посмотрим, что произойдет, если компоновщику не удастся найти подходящие определения. Укажем для этого только два промежуточных объектных файла: main.o и add.o

```
$ gcc add.o main.o
/usr/bin/ld: main.o: in function `main':
main.c:(.text+0x30): undefined reference to `multiply'
collect2: error: ld returned 1 exit status
```

```
$ gcc add.o multiply.o
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/11/../../../x86_64-linux-
gnu/Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
```

План лекции

Раздельная компиляция	Объектные файлы	BLAS, LAPACK, MKL	Работа с простыми числами
30 минут	25 минут	20 минут	15 минут

Введение

Мы рассмотрим 6 разделов

1. Двоичный интерфейс приложений. Здесь мы поговорим об ABI (application binary interface) и его значимости.
2. Форматы объектных файлов. Этот раздел посвящен разным форматам объектных файлов — как актуальным, так и устаревшим.
3. Переносимые объектные файлы. Здесь будут рассмотрены переносимые объектные файлы и самые первые продукты компиляции проекта на языке C.
4. Исполняемые объектные файлы. В этом разделе мы поговорим об исполняемых объектных файлах и объясним, как они создаются из разных переносимых объектных файлов.
5. Статические библиотеки. Этот раздел посвящен статическим библиотекам и их созданию.
6. Динамические библиотеки. Здесь речь пойдет о разделяемых объектных файлах. Будет продемонстрирован процесс их создания из разных переносимых объектных файлов и показан процесс их использования в программе.

Двоичный интерфейс приложений

Любые библиотеки и фреймворки, независимо от используемых в них технологий и языка написания, предоставляют определенный набор возможностей, известный как программный интерфейс приложения (Application Programming Interface, API). Если библиотеку нужно применять в другом коде, то это следует делать через ее API.

API — это публичный интерфейс библиотеки, которого должно быть достаточно для ее полноценного использования; все остальное считается «черным ящиком», внутрь которого нельзя заглянуть.

API — своего рода соглашение, принятое двумя программными компонентами, которые обслуживают или используют друг друга. Концепция ABI имеет похожее назначение, только на другом уровне. Если API обеспечивает совместимость двух программных компонентов с точки зрения их функционального взаимодействия, то ABI гарантирует, что две программы и их соответствующие объектные файлы совместимы на уровне машинных инструкций.

ABI обычно содержит следующую информацию:

- набор инструкций целевой архитектуры, включая инструкции процессора, структуры памяти, порядок следования байтов, регистры и т. д.;
- существующие типы данных, их размеры и правила выравнивания;
- соглашение о вызове функций, включая такие подробности, как структура стекового фрейма и порядок размещения аргументов в стеке;
- механизм системных вызовов в Unix-подобных ОС;
- используемые форматы объектных файлов, включая переносимые, исполняемые и разделяемые;
- если речь идет об объектных файлах, сгенерированных компилятором C++, то в состав ABI также входит метод декорирования имен и структура виртуальной таблицы.

Форматы объектных файлов

В Unix-подобных операционных системах, таких как Linux и BSD, самым распространенным стандартом ABI является System V. В рамках System V ABI по умолчанию используется формат объектных файлов ELF **(Executable and Linking Format — формат выполнения и компоновки).**

На каждой платформе для хранения машинных инструкций используется свой формат объектных файлов. Обратите внимание: речь здесь идет о структуре самих файлов, а не о наборе инструкций, которые поддерживает архитектура.

Форматы, используемые в разных операционных системах.

- ELF применяется в Linux и многих других Unix-подобных ОС.
- Mach-O используется в системах OS X (macOS и iOS).
- PE применяется в Microsoft Windows.

Если обратиться к истории, то можно сказать, что все существующие на сегодняшний день форматы объектных файлов происходят от старого формата a.out, который был разработан для ранних версий Unix. Название расшифровывается как assembler output (вывод ассемблера).

Но вскоре на смену a.out пришел другой формат, COFF (Common Object File Format — общий формат объектных файлов), ставший впоследствии основой для. В рамках выпуска OX/X компания Apple создала свою замену a.out под названием Mach-O. В Windows в качестве формата объектных файлов используется PE (Portable Execution — переносимый исполняемый файл), основанный на COFF.

Более подробную информацию о формате ELF и его структуре можно найти по адресу https://www.uclibc.org/docs/psABI-x86_64.pdf. Обратите внимание: в этом документе идет речь о System V ABI для 64-битной архитектуры AMD (amd64).

Переносимые объектные файлы

Переносимые объектные файлы создаются на этапе компиляции в машинный код в ходе сборки проекта. Они считаются промежуточными и используются в качестве ингредиентов для создания дальнейших и конечных продуктов. В связи с этим мы уделим им дополнительное внимание и изучим их содержимое.

В переносимом объектном файле, полученном из скомпилированной единицы трансляции, можно найти следующие элементы:

- инструкции машинного уровня, сгенерированные из функций, найденных в единице трансляции (код);
- значения инициализированных глобальных переменных, объявленных в единице трансляции (данные);
- таблицу символов, содержащую все символы, которые были объявлены и на которые ссылается единица трансляции.

Переносимые объектные файлы связаны с процессом, в ходе которого компоновщик формирует более крупный объектный файл — исполняемый или разделяемый.

funcs.c

```c
int max(int a, int b) {
    return a > b ? a : b;
}
int max_3(int a, int b, int c) {
    int temp = max(a, b);
    return c > temp ? c : temp;
}
```

main.c

```c
int max(int, int);
int max_3(int, int, int);
int a = 5, b = 10;
int main(int argc, char **argv){
    int m1 = max(a, b);
    int m2 = max_3(5, 8, -1);
    return 0;
}
```

Переносимые объектные файлы

```
$ gcc -c funcs.c -o funcs.o
$ gcc -c main.c -o main.o
$ readelf -hSl funcs.o
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              REL (Relocatable file)
  Machine:                           Advanced Micro Devices X86-64
  ……
  Number of section headers:         13
  Section header string table index: 12

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  [ 0]                   NULL             0000000000000000  00000000
       0000000000000000  0000000000000000           0     0     0
  [ 1] .text             PROGBITS         0000000000000000  00000040
       000000000000004f  0000000000000000  AX       0     0     1
…
  [ 3] .data             PROGBITS         0000000000000000  0000008f
       0000000000000000  0000000000000000  WA       0     0     1
  [ 4] .bss              NOBITS           0000000000000000  0000008f
       0000000000000000  0000000000000000  WA       0     0     1
…
  [10] .symtab           SYMTAB           0000000000000000  00000138
       0000000000000078  0000000000000018          11     3     8
…
```

Содержимое объектного файла funcs.o в формате ELF

Секция **.text** содержит все машинные инструкции для единицы трансляции. В секциях **.data и .bss** находятся соответственно значения для инициализированных глобальных переменных и количество байтов, необходимых для неинициализированных глобальных переменных. Секция **.symtab** хранит таблицу символов.

Переносимые объектные файлы

Таблица символов объектного файла funcs.o

```
$readelf -s funcs.o
Symbol table '.symtab' contains 5 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS funcs.c
     2: 0000000000000000     0 SECTION LOCAL  DEFAULT    1 .text
     3: 0000000000000000    27 FUNC    GLOBAL DEFAULT    1 max
     4: 000000000000001b    52 FUNC    GLOBAL DEFAULT    1 max_3
```

Таблица символов объектного файла main.o

```
$ readelf -s main.o
Symbol table '.symtab' contains 8 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS main.c
     2: 0000000000000000     0 SECTION LOCAL  DEFAULT    1 .text
     3: 0000000000000000     4 OBJECT  GLOBAL DEFAULT    3 a
     4: 0000000000000004     4 OBJECT  GLOBAL DEFAULT    3 b
     5: 0000000000000000    73 FUNC    GLOBAL DEFAULT    1 main
     6: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND max
     7: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND max_3
```

Символы, относящиеся к глобальным переменным a и b, а также символ функции main размещены по адресам, которые не похожи на итоговые. Это признак переносимого объектного файла.

```
$ gcc funcs.o main.o -o a.out
$ readelf -hSl a.out
ELF Header:
  ...
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  ...
Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  [ 0]                   NULL             0000000000000000  00000000
       0000000000000000  0000000000000000           0     0     0
  [ 1] .interp           PROGBITS         0000000000000318  00000318
       000000000000001c  0000000000000000   A       0     0     1
  [ 2] .note.gnu.pr[...] NOTE             0000000000000338  00000338
       0000000000000030  0000000000000000   A       0     0     8
...
  [28] .shstrtab         STRTAB           0000000000000000  000035e5
       000000000000010c  0000000000000000           0     0     1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  D (mbind), l (large), p (processor specific)
Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz            MemSiz              Flags  Align
  PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
                 0x00000000000002d8 0x00000000000002d8  R      0x8
  INTERP         0x0000000000000318 0x0000000000000318 0x0000000000000318
                 0x000000000000001c 0x000000000000001c  R      0x1
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
  ...
                 0x0000000000000000 0x0000000000000000  RW     0x10
  GNU_RELRO      0x0000000000002df0 0x0000000000003df0 0x0000000000003df0
                 0x0000000000000210 0x0000000000000210  R      0x1
 Section to Segment mapping:
  Segment Sections...
   00
   01     .interp
...
   12     .init_array .fini_array .dynamic .got
```

Исполняемые объектные файлы – это один из конечных продуктов компиляции проекта на языке C. Они содержат те же элементы, что и их переносимые аналоги: машинные инструкции, значения для инициализированных глобальных переменных и таблицу символов. Отличается только расположение этих элементов.

Исполняемые объектные файлы

Вывод по ELF предыдущего слайда:

- С точки зрения формата ELF этот объектный файл является разделяемым. Иными словами, в ELF исполняемый объектный файл отличается от разделяемого только наличием определенных сегментов наподобие INTERP. Этот сегмент (который на самом деле ссылается на секцию .interp) используется загрузчиком для запуска и выполнения исполняемого файла.
- Четыре сегмента выделены жирным шрифтом. Первый, INTERP, уже рассмотрен в предыдущем пункте. Второй, TEXT, содержит все секции с машинными инструкциями. В третьем, DATA, находятся все значения, которые будут использоваться для инициализации глобальных переменных и других ранних структур. Четвертый сегмент ссылается на секцию с информацией, которая относится к динамической компоновке, такой как местоположение разделяемых объектных файлов, загружаемых во время выполнения.
- Если сравнивать с переносимым разделяемым объектным файлом, здесь есть больше секций, которые, вероятно, содержат данные, необходимые для загрузки и выполнения программы.

Исполняемые объектные файлы

```
$ readelf -s a.out

Symbol table '.dynsym' contains 6 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND _[...]@GLIBC_2.34 (2)
     2: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _ITM_deregiste
     3: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND __gmon_start__
     4: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _ITM_register
     5: 0000000000000000     0 FUNC    WEAK   DEFAULT  UND [...]@GLIBC_2

Symbol table '.symtab' contains 40 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS Scrt1.o
     2: 000000000000038c    32 OBJECT  LOCAL  DEFAULT    4 __abi_tag
    ...
    21: 0000000000004000     0 NOTYPE  WEAK   DEFAULT   23 data_start
    22: 0000000000001144    52 FUNC    GLOBAL DEFAULT   14 max_3
    23: 0000000000004014     4 OBJECT  GLOBAL DEFAULT   23 b
    24: 0000000000004018     0 NOTYPE  GLOBAL DEFAULT   23 _edata
    25: 00000000000011c4     0 FUNC    GLOBAL HIDDEN    15 _fini
    26: 0000000000001129    27 FUNC    GLOBAL DEFAULT   14 max
    ...
    38: 0000000000000000     0 FUNC    WEAK   DEFAULT  UND __cxa_finalize@G[...]
    39: 0000000000001000     0 FUNC    GLOBAL HIDDEN    11 _init
```

Исполняемый объектный файл содержит две разные таблицы символов. Первая, .dynsym, включает символы, которые должны быть разрешены во время загрузки программы, а во второй, .symtab, находится все то же, что и в динамической таблице, плюс все разрешенные символы.
В исполняемом объектном файле абсолютными являются обычные адреса, а в разделяемом — относительные.

Статические библиотеки

Статические библиотеки — один из возможных продуктов компиляции проекта на языке C.

Статическая библиотека в Unix — обычный архив с переносимыми объектными файлами. Как правило, она компонуется с другими объектными файлами в целях получения итоговой исполняемой программы.

Обратите внимание: сами по себе статические библиотеки не являются объектными файлами. Они скорее служат их контейнерами. Иными словами, это не ELFфайлы в Linux и не Mach-O-файлы в macOS. Это просто архивы, созданные Unix-утилитой ar.

Перед компоновкой из статической библиотеки извлекаются переносимые объектные файлы. Затем компоновщик ищет в них неопределенные символы и пытается их разрешить.

В системах семейства Unix применительно к статическим библиотекам действует общепринятое соглашение об именовании. Имя файла должно начинаться с lib и иметь расширение .a. В разных операционных системах могут действовать разные правила; например, в Microsoft Windows статические библиотеки имеют расширение .lib. Примерно так выглядит создание статической библиотеки из множества переносимых объектных файлов:

```
$ ar crs libexample.a aa.o bb.o ... zz.o
```

Про аргумент crs

Статические библиотеки

Библиотека для вычислительной геометрии пример с ноутбука. Будет доступен по ссылке где и остальной (22_1)

```
$ gcc -c trigon.c -o trigon.o
$ gcc -c 2d.c -o 2d.o
$ gcc -c 3d.c -o 3d.o
$ ar crs libgeometry.a trigon.o 2d.o 3d.o
$ ar t libgeometry.a
trigon.o
2d.o
3d.o
```

- 2d.c
- 2d.o
- 3d.c
- 3d.o
- geometry.h
- libgeometry.a
- main.c
- trigon.c
- trigon.o

Статические библиотеки

main.c

```
#include <stdio.h>
#include "geometry.h"
int main(int argc, char **argv)
{
    cartesian_pos_2d_t cartesian_pos;
    cartesian_pos.x = 100;
    cartesian_pos.y = 200;
    polar_pos_2d_t polar_pos =
        convert_to_2d_polar_pos(&cartesian_pos);
    printf("Polar Position: Length: %f, Theta: %f (deg)\n",
           polar_pos.length, polar_pos.theta);
    return 0;
}
```

```
$ gcc -c main.c -o main.o
$ gcc main.o -
L/mnt/c/Users/Mfili/Desktop/learningc++/imperative_programming/lecti
ons_PowerPoint/lection_22_1 -lgeometry -lm -o a.out
$ ./a.out
Polar Position: Length: 223.606798, Theta: 63.434949 (deg)
```

Статические библиотеки

- Параметр -L/../lesson22_1 сообщает компилятору gcc, что каталог /../lesson22_1 входит в число тех мест, в которых можно найти статические и разделяемые библиотеки. По умолчанию компоновщик ищет библиотечные файлы в традиционных каталогах, таких как /usr/lib или /usr/local/lib. Если не указать параметр -L, то компоновщик выполнит поиск только по этим стандартным путям.
- Параметр -lgeometry говорит компилятору gcc, что ему нужно искать файл libgeometry.a или libgeometry.so. Расширение принадлежит разделяемым библиотекам, о которых мы поговорим в следующем разделе.
- Параметр -lm сообщает компилятору gcc, что нужно искать еще одну библиотеку с именем libm.a или libm.so. Она хранит определения математических функций, доступных в glibc, — в частности, cos, sin и acos. Обратите внимание: мы собираем пример 3.3 на компьютере под управлением Linux, поэтому в качестве реализации стандартной библиотеки C используется glibc. В macOS и, возможно, в других операционных системах данный параметр можно опустить.
- Параметр -o a.out говорит компилятору gcc, что итоговый исполняемый файл должен называться a.out.

Стоит отметить, что после компоновки программа не будет зависеть от наличия статических библиотек, поскольку все их содержимое встраивается в ее исполняемый файл. Иными словами, полученная программа является самостоятельной и для ее запуска не требуется присутствие статической библиотеки.

Динамические библиотеки

Динамические (они же разделяемые) библиотеки — еще один продукт компиляции с возможностью повторного использования. От статических библиотек они отличаются тем, что не входят в состав итогового исполняемого файла. Вместе этого их необходимо загружать и подключать во время запуска процесса.

В динамических библиотеках, неопределенные символы могут оставаться и после работы компоновщика; их поиск начинается в момент, когда программа готовится к загрузке и выполнению. Если у нас есть неопределенные динамические символы, то этап компоновки нужно видоизменить. Во время загрузки исполняемого файла и его подготовки к выполнению в виде процесса используется динамический компоновщик/загрузчик.

Неопределенные динамические символы должны загружаться из разделяемых объектных файлов — близких родственников статических библиотек. В большинстве Unix-подобных систем разделяемые объектные файлы имеют расширение .so, а в macOS — .dylib.

Перед самим запуском процесса разделяемый объектный файл загружается в участок памяти, доступный этому процессу. Данная процедура выполняется динамическим компоновщиком (или загрузчиком), который загружает и выполняет программу.

Во-первых, символы имеют относительные и абсолютные адреса, что позволяет загружать их в рамках сразу нескольких процессов.

Во-вторых, в разделяемых объектных файлах, в отличие от исполняемых, нет сегментов для загрузки программы. Это фактически означает, что разделяемые объектные файлы нельзя выполнять.

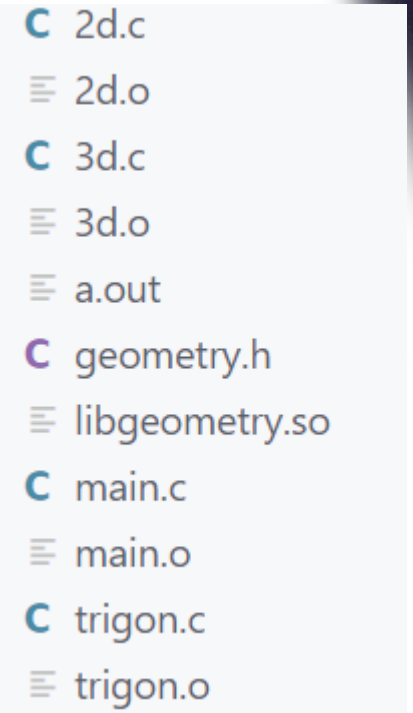
Динамические библиотеки

Дополнительный параметр **-fPIC** обязателен, если вам нужно создать динамическую библиотеку из набора переносимых объектных файлов. **PIC** расшифровывается как position independent code (позиционно независимый код).

Более подробную информацию о том, как работает динамическая загрузка программ. и разделяемых объектных файлов.

Чтобы создать динамическую библиотеку, нам снова нужно будет воспользоваться компилятором (в нашем случае это gcc). В отличие от статической библиотеки, которая является обычным архивом, разделяемый объектный файл остается объектным.

```
$ gcc -c trigon.c -fPIC -o trigon.o
$ gcc -c 2d.c -fPIC -o 2d.o
$ gcc -c 3d.c -fPIC -o 3d.o
$ gcc main.o -
L/mnt/c/Users/Mfili/Desktop/learningc++/imperative_programming/lections_Po
werPoint/lection_22_1 -lgeometry -lm -o a.out
$ ./a.out
./a.out: error while loading shared libraries: libgeometry.so: cannot open
shared object file: No such file or directory
$ export
LD_LIBRARY_PATH=/mnt/c/Users/Mfili/Desktop/learningc++/imperative_programm
ing/lections_PowerPoint/lection_22_1
$ ./a.out
Polar Position: Length: 223.606798, Theta: 63.434949 (deg)
```



Ручная загрузка разделяемых библиотек

```
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>
#include "geometry.h"
polar_pos_2d_t (*func_ptr)(cartesian_pos_2d_t *);
int main(int argc, char **argv) {
    void *handle =
dlopen("/mnt/c/Users/Mfili/Desktop/learningc++/imperative_programming/lect
ions_PowerPoint/lection_22_/libgeometry.so", RTLD_LAZY);
    if (!handle) {
        fprintf(stderr, "%s\n", dlerror());
        exit(1);
    }
    func_ptr = dlsym(handle, "convert_to_2d_polar_pos");
    if (!func_ptr) {
        fprintf(stderr, "%s\n", dlerror());
        exit(1);
    }
    cartesian_pos_2d_t cartesian_pos;
    cartesian_pos.x = 100;
    cartesian_pos.y = 200;
    polar_pos_2d_t polar_pos = func_ptr(&cartesian_pos);
    printf("Polar Position: Length: %f, Theta: %f (deg)\n",
           polar_pos.length, polar_pos.theta);
    return 0;
}
```

Разделяемые объектные файлы можно загружать и использовать иначе, не задействуя загрузчик (динамический компоновщик), который делает это автоматически. Идея в том, что программист загружает динамическую библиотеку вручную непосредственно перед использованием ее символов (функций).

Ручная загрузка разделяемых библиотек

Параметр **-lm** который позволит выполнить компоновку разделяемого объектного файла со стандартной математической библиотекой libm.so. Мы используем его в связи с тем, что в ходе ручной загрузки файла libgeometry.so его зависимости должны загружаться автоматически. В противном случае мы получим сообщения об отсутствии таких символов, как cos или sqrt, которые нужны самой библиотеке libgeometry.so. Обратите внимание: стандартная математическая библиотека не компонуется с итоговым исполняемым файлом; она будет разрешена автоматически во время загрузки libgeometry.so.

```
$ gcc -shared 2d.o 3d.o trigon.o -lm -o libgeometry.so
$ gcc main_2.c -ldl -o a_2.out
$ ./a_2.out
Polar Position: Length: 223.606798, Theta: 63.434949 (deg)
```

План лекции

Раздельная компиляция	Объектные файлы	BLAS, LAPACK, MKL	Работа с простыми числами
30 минут	25 минут	20 минут	15 минут

Общее состояние библиотек BLAS, LAPACK и MKL

Библиотеки линейной алгебры: BLAS, LAPACK, MKL

- ***BLAS (Basic Linear Algebra Subprograms):*** Базовые операции с векторами и матрицами. Оптимизированы по скорости, используются как основа для других библиотек.
- ***LAPACK (Linear Algebra Package):*** Расширяет BLAS, предоставляя алгоритмы для решения систем уравнений, собственных значений и разложений матриц.
- ***MKL (Intel Math Kernel Library):*** Коммерческая реализация BLAS и LAPACK от Intel, оптимизирована для процессоров Intel, включает дополнительные функции (например, FFT).
- Состояние:
  - BLAS и LAPACK — стандарты с открытыми референсными реализациями (Netlib), но для высокой производительности используют оптимизированные версии (OpenBLAS, MKL).
  - MKL — лидер по производительности на Intel CPU, бесплатен для личного использования с 2021 года.
  - Активно применяются в научных вычислениях, машинном обучении и симуляциях.
- Зачем в C?: Интерфейсы на Fortran, но легко интегрируются в C через заголовки (cblas.h) и компиляцию.

Что такое BLAS?

- BLAS — это набор подпрограмм для базовых операций линейной алгебры:
  - Уровень 1: операции с векторами (скалярное произведение, норма).
  - Уровень 2: матрично-векторные операции (умножение матрицы на вектор).
  - Уровень 3: матрично-матричные операции (умножение матриц).
- Реализация:
  - Референсная версия: http://www.netlib.org/blas/
  - Оптимизированная: OpenBLAS (https://github.com/OpenMathLib/OpenBLAS).
- Зачем?: Высокая производительность благодаря оптимизации под архитектуру процессора.

Установка BLAS (OpenBLAS)

OpenBLAS — популярная открытая реализация BLAS.

- Установка на Ubuntu:

```
sudo apt update
sudo apt install libopenblas-dev
```

- • Компиляция из исходников:

  1. Скачать с GitHub:

```
git clone https://github.com/OpenMathLib/OpenBLAS.git
```

  2. Сборка:
  3. cd OpenBLAS
  4. make

```
sudo make install
```

  5. Библиотека обычно устанавливается в /usr/local/lib.

- Проверка: После установки доступны файлы libopenblas.a и libopenblas.so.
- Связывание с C:

```
gcc -o program program.c -I/usr/local/include -L/usr/local/lib -lopenblas
```

Пример кода с BLAS (уровень 1)

Задача: Вычислить скалярное произведение двух векторов с помощью cblas_ddot.

```
#include <stdio.h>
#include <cblas.h>
int main()
{
    int n = 3;
    double x[] = {1.0, 2.0, 3.0};
    double y[] = {4.0, 5.0, 6.0};
    double result;
    // Скалярное произведение: result = x^T * y
    result = cblas_ddot(n, x, 1, y, 1);
    printf("Скалярное произведение: %.2f\n", result); // Ожидаем 32.0
    return 0;
}
```

```
$ gcc main.c -o main –lopenblas
$ ./main
Скалярное произведение: 32.00
```

Пример кода с BLAS (уровень 3)

Задача: Умножить две матрицы A (2x3) и B (3x2) с помощью cblas_dgemm.

```
#include <stdio.h>
#include <cblas.h>
int main()
{
    double A[] = {1, 2, 3, 4, 5, 6};    // 2x3
    double B[] = {7, 8, 9, 10, 11, 12}; // 3x2
    double C[4] = {0};                  // 2x2
    double alpha = 1.0, beta = 0.0;
    cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
                2, 2, 3, alpha, A, 3, B, 2, beta, C, 2);
    printf("C = [%f, %f; %f, %f]\n", C[0], C[1], C[2], C[3]);
    return 0;
}
```

```
$ gcc main.c -o main -lopenblas
$ ./main
C = [58.000000, 64.000000; 139.000000, 154.000000]
```

История BLAS

Происхождение:

Разработан в 1970-х годах (BLAS Level 1 — 1979, авторы: Чарльз Лоусон и др.).

Цель: стандартизировать базовые операции для научных вычислений на Fortran.

Эволюция:

Level 2 (1988): матрично-векторные операции.

Level 3 (1990): матрично-матричные операции для суперкомпьютеров.

Интересный факт:

BLAS изначально был написан на Fortran, но его идеи настолько универсальны, что он стал основой для GPU-библиотек вроде cuBLAS (NVIDIA).

Современность:

Используется в машинном обучении (TensorFlow, PyTorch) через оптимизированные реализации (MKL, OpenBLAS).

Актуальность:

BLAS остаётся актуальной, потому что её идеи универсальны: она решает базовые задачи линейной алгебры, которые лежат в основе почти всех вычислений — от физики до машинного обучения. Её простота и стандартизация позволяют легко адаптировать код под новые архитектуры, а оптимизированные реализации, такие как OpenBLAS или MKL, обеспечивают высокую производительность даже на современных процессорах и GPU. К тому же, BLAS изначально проектировался с учётом переносимости — код на Fortran работал на любых машинах 1970-х, а сегодня его просто "ускоряют" под новые технологии."

Еще о BLAS

Почему BLAS быстр?
Использует кэш-память процессора эффективно.
Пример: *умножение матриц (Level 3) разбивается на блоки, Блочная версия минимизирует обращения к памяти.*
Реальный пример:
Референсный BLAS: ~10 MFLOPS (миллионов операций в секунду).
OpenBLAS на современном CPU: до 100 GFLOPS (гигафлопс).
Трюк оптимизации: Векторные инструкции (SIMD): AVX, SSE в OpenBLAS/MKL позволяют обрабатывать 4–8 чисел за один цикл.
Тест производительности (22_1 пример):
$ ./main
Чистый C: 8.300392 сек
BLAS: 0.828575 сек
Связь с ML: Операции с тензорами в нейронных сетях сводятся к умножению матриц (BLAS Level 3). Пример: свёртка в CNN → преобразование в dgemm.
Реальный кейс: PyTorch использует BLAS через MKL или OpenBLAS для ускорения обучения моделей.
Интересно:
NVIDIA cuBLAS переносит BLAS на GPU, увеличивая скорость в 10–100 раз.
OpenBLAS адаптируется под ARM (например, Apple M1/M2).

Что такое LAPACK?

LAPACK (Linear Algebra Package) — библиотека для сложных задач линейной алгебры:

- Решение систем линейных уравнений (Ax = b).
- Разложение матриц (LU, QR, SVD).
- Вычисление собственных значений и векторов.

Пример: Решение Ax = b через LU-разложение:

Реализация:

Референсная версия: http://www.netlib.org/lapack/

Оптимизированная: MKL, OpenBLAS (с LAPACK внутри).

Интересное про LAPACK — связь с BLAS

LAPACK делегирует базовые операции (умножение матриц, сложение) BLAS.

Эффективность LAPACK зависит от скорости BLAS (Level 3).

Пример оптимизации: LU-разложение в dgesv использует dgemm (BLAS) для матричных операций:

A = P * L * U, где P — матрица перестановок

OpenBLAS/MKL ускоряют dgemm, ускоряя весь LAPACK.

Интересный факт: LAPACK изначально писался на Fortran 77 (1992 год), но до сих пор актуален благодаря BLAS и новым реализациям (LAPACKE для C).

Установка LAPACK

OpenBLAS включает LAPACK, что упрощает установку.

Установка на Ubuntu:

```
sudo apt update
sudo apt install liblapacke-dev
```

- • Компиляция из исходников:

  1. Скачать с GitHub:

```
git clone https://github.com/OpenMathLib/OpenBLAS.git
```

  2. Сборка:
  3. cd OpenBLAS
  4. make

```
make USE_OPENMP=1 LAPACK=1
```

  5. Библиотека обычно устанавливается в /usr/local/lib.

- Проверка: После установки доступны файлы libopenblas.a и libopenblas.so.
- Связывание с C:

```
gcc -o program program.c -I/usr/local/include -L/usr/local/lib -lopenblas
```

Пример кода с LAPACK

Задача: Решить систему Ax = b с помощью dgesv (LU-разложение).

```c
#include <stdio.h>
#include <lapacke.h>
int main() {
    lapack_int n = 3, nrhs = 1, lda = 3, ldb = 3, info;
    lapack_int ipiv[3];                          // Для перестановок
    double A[9] = {4, 1, 1, 2, 5, 2, 1, 1, 3}; // Матрица 3x3
    double b[3] = {1, 2, 3};                     // Вектор b
    // Решаем систему
    info = LAPACKE_dgesv(LAPACK_ROW_MAJOR, n, nrhs, A, lda, ipiv, b, ldb);
    if (info == 0)
        printf("Решение: x = [%f, %f, %f]\n", b[0], b[1], b[2]);
    else
        printf("Ошибка: %d\n", info);
    return 0;
}
```

```
$ gcc main.c -o main -llapacke -lopenblas
$ ./main
Собственные значения: [2.000000, 3.267949, 6.732051]
```

Пример 2 кода с LAPACK

Задача: Найти собственные значения матрицы с помощью dsyev (симметричная матрица).

```
#include <stdio.h>
#include <lapacke.h>
int main() {
    lapack_int n = 3, lda = 3, info;
    double A[9] = {5, 2, 1, 2, 4, 0, 1, 0, 3}; // Симметричная 3x3
    double w[3];                               // Собственные значения
    info = LAPACKE_dsyev(LAPACK_ROW_MAJOR, 'N', 'U', n, A, lda, w);
    if (info == 0)
        printf("Собственные значения: [%f, %f, %f]\n", w[0], w[1], w[2]);
    else
        printf("Ошибка: %d\n", info);
    return 0;
}
```

```
$ gcc main.c -o main -llapacke -lopenblas
$ ./main
Решение: x = [0.222222, 2.000000, 3.000000]
```

Что такое Intel MKL?

MKL (Math Kernel Library) — библиотека от Intel для высокопроизводительных вычислений:

- Включает BLAS и LAPACK с оптимизацией под процессоры Intel.
- Дополнительно: FFT (быстрое преобразование Фурье), sparse-операции, генерация случайных чисел.

Особенности:

- Поддержка многопоточности (OpenMP) и векторных инструкций (AVX, AVX-512).
- Интеграция с C, C++, Fortran, Python.

Интересное про MKL

Преимущества:

- Оптимизация под Intel CPU (AVX-512 даёт до 2x ускорения).
- Поддержка GPU (oneAPI) для будущих расширений.
- Интеграция с инструментами Intel (VTune, Advisor).

Интересный факт:

MKL автоматически выбирает лучший алгоритм в зависимости от размера матрицы и процессора.

Сравнение производительности:

OpenBLAS vs MKL на Intel Core i7: MKL до 20% быстрее для больших матриц.

Установка Intel MKL

Установка MKL на Ubuntu

Установка через apt:

Добавьте GPG-ключ:

wget -O- https://apt.repos.intel.com/intel-gpg-keys/GPG-PUB-KEY-INTEL-SW-PRODUCTS.PUB | gpg --dearmor | sudo tee /usr/share/keyrings/oneapi-archive-keyring.gpg > /dev/null

Добавьте репозиторий:

echo "deb [signed-by=/usr/share/keyrings/oneapi-archive-keyring.gpg] https://apt.repos.intel.com/oneapi all main" | sudo tee /etc/apt/sources.list.d/oneAPI.list

Обновите и установите:

sudo apt update

sudo apt install intel-oneapi-mkl

sudo apt install intel-oneapi-mkl-devel

Настройте окружение:

source /opt/intel/oneapi/setvars.sh

Проверка: Библиотеки в /opt/intel/oneapi/mkl/latest/lib.

Связывание с C:

gcc -o program program.c -I/opt/intel/oneapi/mkl/latest/include -L/opt/intel/oneapi/mkl/latest/lib/intel64 -lmkl_intel_lp64 -lmkl_core -lmkl_sequential -lpthread -lm

Пример кода с MKL (BLAS)

Задача: Умножить матрицы A и B с помощью cblas_dgemm.

```
#include <stdio.h>
#include <mkl.h>
int main()
{
    double A[] = {1, 2, 3, 4, 5, 6};    // 2x3
    double B[] = {7, 8, 9, 10, 11, 12}; // 3x2
    double C[4] = {0};                  // 2x2
    double alpha = 1.0, beta = 0.0;
    cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
                2, 2, 3, alpha, A, 3, B, 2, beta, C, 2);
    printf("C = [%f, %f; %f, %f]\n", C[0], C[1], C[2], C[3]);
    return 0;
}
```

```
$ gcc main.c -o main -I/opt/intel/oneapi/mkl/latest/include -L/opt/intel/oneapi/mkl/latest/lib/intel64 -lmkl_intel_lp64 -lmkl_core -lmkl_sequential -lpthread -lm
$ ./main
C = [58.000000, 64.000000; 139.000000, 154.000000]
```

Пример кода с MKL (LAPACK)

Задача: Решить систему Ax = b с помощью dgesv.

```c
#include <stdio.h>
#include <mkl.h>
int main() {
    MKL_INT n = 3, nrhs = 1, lda = 3, ldb = 3, info;
    MKL_INT ipiv[3];
    double A[9] = {4, 1, 1, 2, 5, 2, 1, 1, 3};
    double b[3] = {1, 2, 3};
    info = LAPACKE_dgesv(LAPACK_ROW_MAJOR, n, nrhs, A, lda, ipiv, b, ldb);
    if (info == 0)
        printf("Решение: x = [%f, %f, %f]\n", b[0], b[1], b[2]);
    else
        printf("Ошибка: %d\n", info);
    return 0;
}
```

```
$ gcc main.c -o main -I/opt/intel/oneapi/mkl/latest/include -L/opt/intel/oneapi/mkl/latest/lib/intel64 -lmkl_intel_lp64 -lmkl_core -lmkl_sequential -lpthread -lm
$ ./main
Решение: x = [0.222222, 2.000000, 3.000000]
```

План лекции

Раздельная компиляция	Объектные файлы	BLAS, LAPACK, MKL	Работа с простыми числами
30 минут	25 минут	20 минут	15 минут

Работа с простыми числами

В 1640 году Ферма предположил, что формула

$$F_n = 2^{2^n} + 1$$

всегда дает простые числа, и числа этого вида получили название "числа Ферма". Предположение Ферма справедливо для n от 0 до 4, но в 1732 году Эйлер обнаружил, что

$$F_5 = 2^{2^5} + 1 = 641 \cdot 6\,700\,417.$$

(Мы уже имели дело с этими множителями, когда рассматривали вопросы деления на константу на 32-битовом компьютере.) Затем в 1880 году Ландри показал, что

$$F_6 = 2^{2^6} + 1 = 274\,177 \cdot 67\,280\,421\,310\,721.$$

В настоящее время известно, что F - составные числа для многих больших значений n, в частности для и от 7 до 16 включительно. В настоящее время неизвестно ни одно значение n > 4, для которого число Ферма было бы простым. Увы, гипотеза оказалась слишком поспешной…

Почему же Ферма использовал двойное возведение в степень? Он знал, что если ты имеет нечетный множитель, отличный от 1, то 2^m +1 -- составное число. Если m = ab, где b нечетно и не равно 1, то

$$2^{ab} + 1 = (2^a + 1)(2^{a(b-1)} - 2^{a(b-2)} + 2^{a(b-3)} - \cdots + 1).$$

Зная это, Ферма заинтересовался числами 2^m +1, такими, где m не имеет ни одного нечетного множителя, отличного от 1, т.е. m = 2^n. Он испробовал несколько значений n и убедился, что они дают простые числа 2^n + 1.

Работа с простыми числами

В 1772 году Эйлером был открыт удивительный полином

$$f(n) = n^2 + n + 41,$$

обладающий тем свойством, что он дает простые числа для всех и от 0 до 39.

Этот результат может быть расширен. Так как

$$f(-n) = n^2 - n + 41 = f(n-1),$$

f(-n) дает простые числа для каждого n из диапазона от 1 до 40, т.е. f(n) дает простые числа для всех и от -1 до -40. Таким образом, полином

$$f(n-40) = (n-40)^2 + (n-40) + 41 = n^2 - 79n + 1601$$

дает простые числа для всех n от 0 до 79. (Однако эстетическая привлекательность формулы при этом теряется, так как она дает повторяющиеся и немонотонные результаты: для n = 0,1, …,79 формула дает числа 1601, 1523, 1447, …, 43, 41, 41, 43, …, 1447, 1523, 1601.)

Несмотря на всю привлекательность полинома, открытого Эйлером, известно, что не существует полиномиальной формулы f(n), которая давала бы простое число для каждого n (не считая тривиального случая константного полинома типа f(n) = 5). Более того, справедлива следующая теорема.

Работа с простыми числами

Теорема. Если $f(n) = P(n, 2^n, 3^n, \dots, k^n)$ представляет собой полином с целочисленными коэффициентами от своих аргументов и f(n)при n→∞, то f(n) является составным значением для бесконечного множества значений n.

Следовательно, формула типа $n^2 \cdot 2^n + 2n^3 + 2n + 5$ должна давать бесконечное количество составных чисел. С другой стороны, теорема ничего не говорит о формулах, содержащих члены наподобие 2^{2^n}, n^n или n!.

Работа с простыми числами

Формула для n-го целого простого числа может быть получена с помощью функции получения максимального целого, не превосходящего данное число ("пол"), и магического значения
a = 0.20300500070001100001 3…

Число a в десятичной системе счисления представляет собой первое простое число, записанное в первой позиции после десятичной точки, второе простое число, записанное в следующих двух позициях, третье, записанное в следующих трех позициях, и т.д. При этом для очередного простого числа всегда найдется достаточно свободного места, так как $p_n < 10^n$. Не доказывая этого, просто заметим: поскольку известно, что между n и 2n (n≥2) всегда есть простое число, значит, между и n и 10n оно есть наверняка, а отсюда следует, что $p_n < 10^n$. Формула для n-го простого числа выглядит следующим образом:

$$p_n = \left[10^{\frac{n^2+n}{2}} a\right] - 10^n \left[10^{\frac{n^2-n}{2}} a\right]$$

где было использовано соотношение

$$\sum_{i=1}^{n} n = \frac{n^2+n}{2}.$$

Например:

$$p_3 = [10^6 a] - 10^3\,[10^3 a] = 203005 - 203000 = 5.$$

Это довольно дешевый трюк, который требует знания окончательного результата для корректного определения a.

Работа с простыми числами

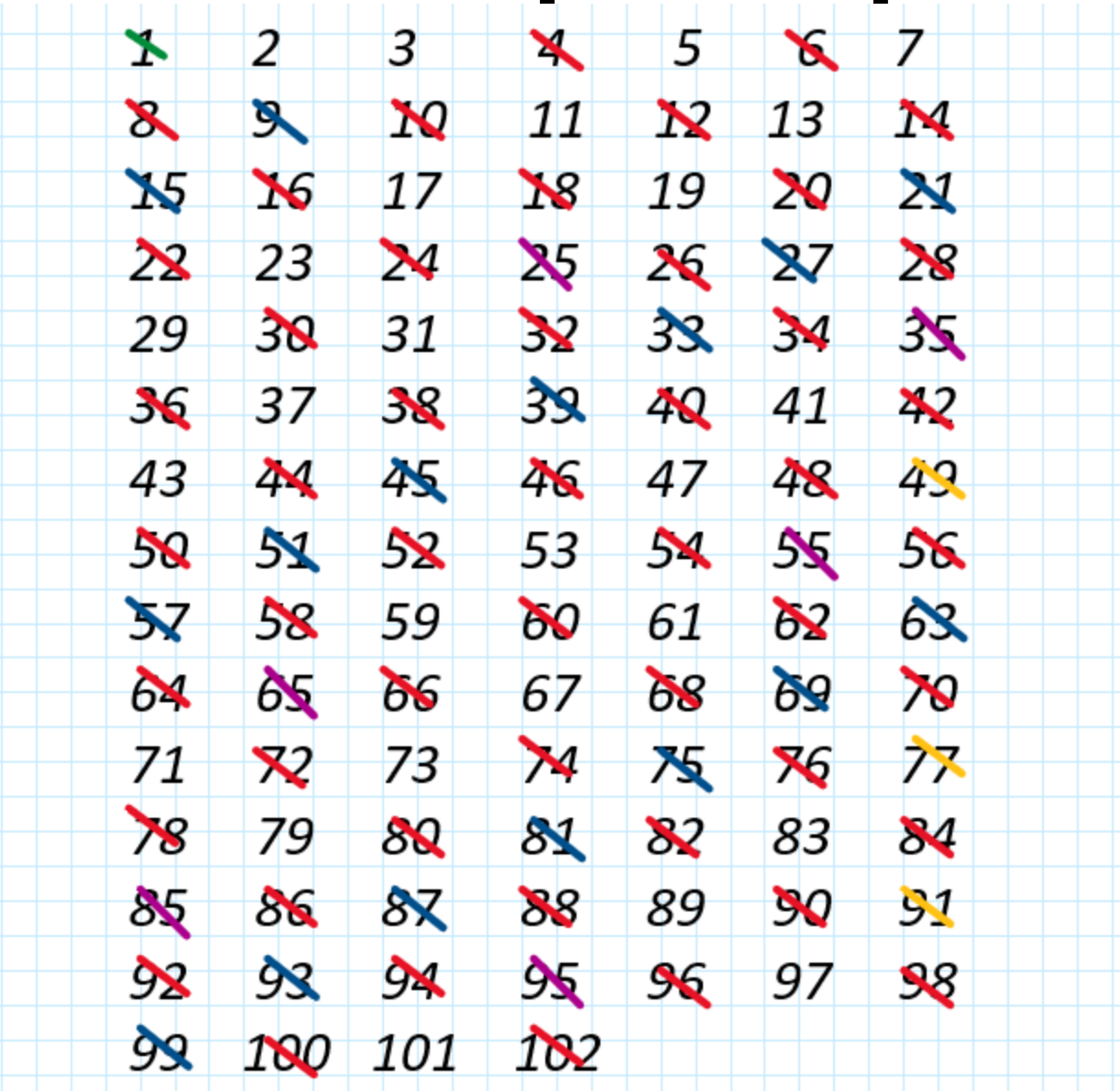
Кристофер.П. Вилланс (C.P. Willans) предложил несколько формул для вычисления простых чисел:

$$p_n = 1 + \sum_{m=2}^{2^n} \left[\sqrt[m]{n} \left(\sum_{x=1}^{m} \left[\cos^2 \pi \frac{(x-1)!+1}{x} \right] \right)^{-\frac{1}{n}} \right]$$

$$\pi(x) = \sum_{k=2}^{x} \left[\cos^2 \left(\pi \frac{(x-1)!+1}{x} \right) \right],$$

где π(x)— количество простых чисел, не превышающих x. Формулы Уилланса математически корректны и всегда работают для вычисления простых чисел, но они крайне неэффективны на практике из-за огромного числа операций.

Решето Эратосфена (Сегментированное)



Временная сложность: O(nloglogn), что делает алгоритм одним из самых быстрых для поиска простых чисел в диапазоне.

Пространственная сложность: O(n) или (O(√n) – сегментированное)

Пол Притчард (Paul Pritchard) разработал несколько алгоритмов для поиска простых чисел, включая так называемое "решето Притчарда" (Sieve of Pritchard) или "динамическое колесное решето". Я опишу его наиболее известный алгоритм — колесное решето — и его сложность.

Временная сложность: O(n/loglogn). Это сублинейная асимптотика, что делает его быстрее, чем O(nloglogn) решета Эратосфена, за счёт минимизации операций удаления составных чисел.

Пространственная сложность: O(n/loglogn) бит в базовой версии, но Притчард предложил вариант с O(n/loglogn) бит, если хранить только колесо и небольшие буферы.